A. Informasi & Ringkasan

1. Identitas

NIM	11S25038
Nama	EVAN BENTA PURBA
Kelas	11IF-2
Judul Praktikum	List of List

2. Capaian & Ringkasan Pemahaman

Hasil Capaian setelah mengikuti praktikum:

- A (Penyelesaian): Selesai (1), Tidak Selesai (0).
- B (Pemahaman): Tidak Paham (1), Kurang Paham (2), Cukup Paham (3), Paham (4), Sangat Paham (5).

no	Kriteria	A	B
1	Mengimplementasikan Tipe List_of_list	1	5

3.Foto Praktikan



B. Penilaian Observasi

Setiap Mahasiswa yang mengikuti kegiatan praktikum perlu melakukan penilaian terhadap diri sendiri dan rekan kerja berdasarkan [Indikator Penilaian Observasi Praktikum].

1. Penilaian Terhadap Diri Sendiri

No	Indikator	Skala (1 - 5)
1	Kepatuhan Terhadap Prosedur	5
2	Kemampuan Penyelesaian Tugas	5
3	Kerja Sama Tim	4
4	Inisiatif dan Problem Solving	5
5	Kedisiplinan dan Etika Kerja	4

C. Laporan Aktivitas Praktikum

1. Mengimplementasikan Tipe List of list

1. Konstruktor

a) konso(e, l), menghasilkan sebuah list dari l n eda, dengan e sebagai elemen terakhir.

b) konsb(l, e), menghasilkan sebuah list dari l dan e, dengan e sebagai elemen terkair list.

2. Selektor

a) firstElmt(l), mengembalikan elemen pertama list l.

b) tailList(l), mengembalikan list tanpa elemen pertama list l.

c) lastElmt(l), mengembalikan elemen terakhir list l.

d) headList(l), mengembalikan list tanpa elemen terakhir list l.

e) midList(l), menghasilkan list untuk elemen yang berada di tengah list l.

3. Operator

a) nbElmt(l), menghitung jumlah elemen list, nol jika kosong.

b) konkat(l1, l2), menghasilkan konkatenasi 2 buah list, dengan list l2 sesudah list l1.

c) reverseList(l), memberikan list yang urutan elemennya adalah kebalikan dari list asal.

d) maxList(l), menghasilkan nilai elemen list yang mempunyai nilai terbesar. Minimal list terdiri atas 1 elemen.

e) elmtKeN(l, n) untuk mengembalikan nilai elemen pada posisi n di list l. Ukuran list harus lebih besar atau sama dengan n.

f) sumList(l), menghasilkan total nilai dari elemen yang terdapat pada list l.

g) averageList(l), menghasilkan nilai rata-rata dari elemen yang terdapat pada list l.

4. Predikat

a) isEmpty(l), adalah benar jika list kosong.

b) isOneElemt(l), adalah benar jika list berisi satu elemen.

c) isEqual(l1, l2), adalah benar jika list l1 sama dengan list l2.

d) isMember(l, x) adalah benar jika x adalah elemen list l.

e) isFirst(l, x) adalah benar jika x adalah elemen pertama pada list l.

f) islast(l, x) adalah benar jika x adalah elemen terakhir pada list l.

g) isNbElemt(l1, l2) adalah benar jika jumlah elemen list l1 sama dengan jumlah elemen list l2.

h) isReverse(l1, l2) adalah benar jika list l2 adalah reverse dari list l1.

i) isXElmtKeN(l, x, n) adalah benar jika elemen x berada pada list l dan tepat berada di posisi n.

Keywords yang dilarang digunakan:

1) length : dipergunakan untuk mengambil panjang elemen. Alternatif silahkan implementasikan sendiri pada fungsi nbElmt dengan menggunakan rekursif.

2) ++ : untuk menggabungkan 2 buah list. Hanya boleh digunakan

pada fungsi main.

3) maximum : untuk mengambil nilai max dari list.

4) !! : untuk mengambil elemen pada posisi tertentu di list. Alternatif silahkan implementasikan sendiri pada fungsi elmtKeN.

Code:

ifs25038-pemfung-praktikum-11 > List_of_list.hs

```haskell
konso e l = e : l

konsb l e = balikList (konso e (balikList l))

prtmElmt (x:_) = x
prtmElmt [] = error "List kosong"

scndList [] = []
scndList (_:xs) = xs

trkhrElmt [] = error "List kosong"
trkhrElmt [x] = x
trkhrElmt (_:xs) = trkhrElmt xs

kplList [] = []
kplList l = balikList (scndList (balikList l))

tnghList [] = []
tnghList [x] = [x]
tnghList l =
  let panjang = hitungPanjang l
      tengah = panjang `div` 2
  in [x | (i, x) <- zip [0..] l, genap panjang && i >= tengah - 1 && i <= tengah || ganjil panjang && i == tengah]

hitungPanjang [] = 0
hitungPanjang (_:xs) = 1 + hitungPanjang xs

nbElmt [] = 0
nbElmt (_:xs) = 1 + nbElmt xs

konkat [] l2 = l2
konkat (x:xs) l2 = x : konkat xs l2

balikList [] = []
balikList (x:xs) = balikList xs `konkat` [x]

maxList [x] = x
maxList (x:xs) = if x > maxList xs then x else maxList xs

elmtKeN (x:_) 1 = x
elmtKeN (_:xs) n = elmtKeN xs (n - 1)
elmtKeN _ _ = error "Posisi tidak valid"

sumList [] = 0
sumList (x:xs) = x + sumList xs

averageList [] = error "List kosong"
averageList l = let s = sumList l
                    len = nbElmt l
                in fromIntegral s / fromIntegral len
```

```haskell
isEmpty [] = True
isEmpty _ = False

isOneElmt l = nbElmt l == 1

isEqual l1 l2 = l1 == l2

isMember [] _ = False
isMember (x:xs) y = x == y || isMember xs y

isFirst [] _ = False
isFirst (x:_) y = x == y

islast [] _ = False
islast [x] y = x == y
islast (_:xs) y = islast xs y

isNbElemt l1 l2 = nbElmt l1 == nbElmt l2

isReverse l1 l2 = l1 == balikList l2

isXElmtKeN l x n = n >= 1 && n <= nbElmt l && elmtKeN l n == x

genap n = n `mod` 2 == 0
ganjil n = n `mod` 2 == 1

getListFromInput 0 = return []
getListFromInput n = do
  x <- readLn :: IO Int
  xs <- getListFromInput (n - 1)
  return (x:xs)

main :: IO ()
main = do
  totalCure1 <- readLn :: IO Int
  lol1 <- getListFromInput totalCure1

  totalCure2 <- readLn :: IO Int
  lol2 <- getListFromInput totalCure2

  ate1  <- readLn :: IO Int
  nerd1 <- readLn :: IO Int
  nerd2 <- readLn :: IO Int
  eks1 <- readLn :: IO Int
  eks2 <- readLn :: IO Int
```

```
let lol1konso = konso ate1 lol1
    lol1konsb = konsb lol1 ate1
    lol2konso = konso ate1 lol2
    lol2konsb = konsb lol2 ate1

putStrLn $ "e = " ++ show ate1
putStrLn $ "l1 = " ++ show lol1
putStrLn $ "l1 (konso) = " ++ show lol1konso
putStrLn $ "l1 (konsb) = " ++ show lol1konsb
putStrLn $ "l2 = " ++ show lol2
putStrLn $ "l2 (konso) = " ++ show lol2konso
putStrLn $ "l2 (konsb) = " ++ show lol2konsb
putStrLn $ "n1 = " ++ show nerd1
putStrLn $ "n2 = " ++ show nerd2
putStrLn $ "x1 = " ++ show eks1
putStrLn $ "x2 = " ++ show eks2
putStrLn $ "firstElmt(l1) = " ++ show (prtmElmt lol1)
putStrLn $ "firstElmt(l2) = " ++ show (prtmElmt lol2)
putStrLn $ "tailList(l1) = " ++ show (scndList lol1)
putStrLn $ "tailList(l2) = " ++ show (scndList lol2)
putStrLn $ "lastElmt(l1) = " ++ show (trkhrElmt lol1)
putStrLn $ "lastElmt(l2) = " ++ show (trkhrElmt lol2)
putStrLn $ "headList(l1) = " ++ show (kplList lol1)
putStrLn $ "headList(l2) = " ++ show (kplList lol2)
putStrLn $ "midList(l1) = " ++ show (tnghList lol1)
putStrLn $ "midList(l2) = " ++ show (tnghList lol2)
putStrLn $ "nbElmt(l1) = " ++ show (nbElmt lol1)
putStrLn $ "nbElmt(l2) = " ++ show (nbElmt lol2)
putStrLn $ "konkat(l1, l2) = " ++ show (konkat lol1 lol2)
putStrLn $ "maxList(l1) = " ++ show (maxList lol1)
putStrLn $ "maxList(l2) = " ++ show (maxList lol2)
putStrLn $ "elmtKeN(l1, n1) = " ++ show (elmtKeN lol1 nerd1)
putStrLn $ "elmtKeN(l2, n2) = " ++ show (elmtKeN lol2 nerd2)
putStrLn $ "sumList(l1) = " ++ show (sumList lol1)
putStrLn $ "sumList(l2) = " ++ show (sumList lol2)
putStrLn $ "averageList(l1) = " ++ show (averageList lol1)
putStrLn $ "averageList(l2) = " ++ show (averageList lol2)
putStrLn $ "isEqual(l1, l2) = " ++ show (isEqual lol1 lol2)
putStrLn $ "isEqual(l1, l2) = " ++ show (isEqual lol1 lol2)
putStrLn $ "isMember(l1, x1) = " ++ show (isMember lol1 eks1)
putStrLn $ "isMember(l2, x2) = " ++ show (isMember lol2 eks2)
putStrLn $ "isFirst(l1, x1) = " ++ show (isFirst lol1 eks1)
putStrLn $ "isFirst(l2, x2) = " ++ show (isFirst lol2 eks2)
putStrLn $ "islast(l1, x1) = " ++ show (islast lol1 eks1)
putStrLn $ "islast(l2, x2) = " ++ show (islast lol2 eks2)
putStrLn $ "isNbElemt(l1, l2) = " ++ show (isNbElemt lol1 lol2)
putStrLn $ "isReverse(l1, l2) = " ++ show (isReverse lol1 lol2)
putStrLn $ "isXElmtKeN(l1, x1, n1) = " ++ show (isXElmtKeN lol1 eks1 nerd1)
putStrLn $ "isXElmtKeN(l2, x2, n2) = " ++ show (isXElmtKeN lol2 eks2 nerd2)
```

Terminal:

Compile:

```
C:\file kuliah ;p\pemfung\pemfunglagi\pemfung praktikum>cd ifs25038-pemfung-praktikum-11

C:\file kuliah ;p\pemfung\pemfunglagi\pemfung praktikum\ifs25038-pemfung-praktikum-11>ghc -o List_of_list List_of_list.hs
[1 of 2] Compiling Main             ( List_of_list.hs, List_of_list.o )
[2 of 2] Linking List_of_list.exe
```

Run:

Input:3123312342233

```
e = 4
l1 = [1,2,3]
l1 (konso) = [4,1,2,3]
l1 (konsb) = [1,2,3,4]
l2 = [1,2,3]
l2 (konso) = [4,1,2,3]
l2 (konsb) = [1,2,3,4]
n1 = 2
n2 = 2
x1 = 3
x2 = 3
firstElmt(l1) = 1
firstElmt(l2) = 1
tailList(l1) = [2,3]
tailList(l2) = [2,3]
lastElmt(l1) = 3
lastElmt(l2) = 3
headList(l1) = [1,2]
headList(l2) = [1,2]
midList(l1) = [2]
midList(l2) = [2]
nbElmt(l1) = 3
nbElmt(l2) = 3
konkat(l1, l2) = [1,2,3,1,2,3]
maxList(l1) = 3
maxList(l2) = 3
elmtKeN(l1, n1) = 2
elmtKeN(l2, n2) = 2
sumList(l1) = 6
sumList(l2) = 6
averageList(l1) = 2.0
averageList(l2) = 2.0
isEqual(l1, l2) = True
isEqual(l1, l2) = True
isMember(l1, x1) = True
isMember(l2, x2) = True
isFirst(l1, x1) = False
isFirst(l2, x2) = False
islast(l1, x1) = True
islast(l2, x2) = True
isNbElemt(l1, l2) = True
isReverse(l1, l2) = False
isXElmtKeN(l1, x1, n1) = False
isXElmtKeN(l2, x2, n2) = False
```